

Gerador e Verificador de Assinaturas

Evelyn Caroline Morais Targino
Departamento de Ciência da Computação
Universidade de Brasília
Brasília, Brasil
221006404

Leonardo Marques de Paula Junior
Departamento de Engenharia Elétrica
Universidade de Brasília
Brasília, Brasil
221029015

Lucas Vasconcelos Pirani Carneiro
Departamento de Ciência da Computação
Universidade de Brasília
Brasília, Brasil
251031785

Vinícius Geraldo dos Santos Guerra
Departamento de Ciência da Computação
Universidade de Brasília
Brasília, Brasil
190096454

Abstract—One of the main topics studied in computer security is the development of mechanisms capable of ensuring the confidentiality, integrity, and authenticity of digital information. In this context, the main objective of this work was to develop an RSA signature generator and verifier for files, enabling the detection of unauthorized changes in content.

Index Terms—Segurança Computacional, RSA, OAEP, Assinaturas Digitais, Gerador de Assinaturas Digitais, Verificador de Assinaturas Digitais

I. INTRODUÇÃO

Um dos principais assuntos estudados em segurança computacional é o desenvolvimento de mecanismos capazes de garantir a confidencialidade, a integridade e a autenticidade de informações digitais. Nesse contexto, o objetivo principal deste trabalho foi desenvolver um gerador e verificador de assinaturas RSA em arquivos, permitindo identificar alterações indevidas no conteúdo.

Este relatório está organizado da seguinte forma: na Seção II são apresentados os fundamentos teóricos necessários; na Seção III é descrito o ambiente experimental e a análise dos resultados; e, por fim, na Seção IV são apresentadas as conclusões obtidas com a implementação.

II. FUNDAMENTAÇÃO TEÓRICA

A. Assinatura Digital

Uma assinatura digital é uma técnica criptográfica utilizada para associar um documento digital ao seu autor, ou para indicar que determinado usuário concorda com o conteúdo desse documento [1]. Para que uma assinatura digital seja eficaz, ela deve possuir duas propriedades fundamentais: ser verificável, isto é, permitir comprovar que o documento foi assinado por um indivíduo específico; e ser não falsificável, de modo que apenas o detentor da chave privada correspondente consiga produzir uma assinatura válida [1].

O funcionamento básico de uma assinatura digital baseia-se na criptografia de chave pública. Nesse modelo, o remetente utiliza sua chave privada para gerar a assinatura, e o destinatário utiliza a chave pública correspondente para verificá-la

[1]. Como as duas chaves estão matematicamente relacionadas, a validação correta da assinatura fornece evidência de autoria e integridade [1].

Na prática, as assinaturas digitais não são aplicadas diretamente sobre o documento inteiro, pois isso seria computacionalmente custoso. Em vez disso, calcula-se um resumo criptográfico da mensagem por meio de uma função hash, e esse resumo é então assinado [1]. Durante a verificação, o receptor calcula novamente o hash da mensagem recebida e compara esse valor com o hash recuperado da assinatura. Se ambos coincidirem, conclui-se que a mensagem não foi alterada e que a assinatura é válida [1].

Além de autenticar o remetente, a assinatura digital também garante a integridade da mensagem, já que qualquer modificação no conteúdo produz um hash diferente e invalida a verificação [1].

B. Algoritmo RSA (Rivest, Shamir e Adleman)

O RSA é um dos algoritmos de criptografia assimétrica mais utilizados. Sua segurança baseia-se na premissa de que, embora seja fácil multiplicar dois números primos grandes para gerar um produto, é extremamente difícil realizar a operação inversa, ou seja, fatorar esse produto de volta em seus componentes primos originais [1].

No algoritmo a mensagem apenas por padrão de bits, que também pode ser representado de forma única por um número inteiro, ou seja, criptografar uma mensagem é equivalente a criptografar um número [1].

A geração da chave é feita da seguinte maneira:

- 1) Escolher dois números primos grandes p e q , 1024 bits cada, por exemplo.
- 2) Calcular $n = pq$ e $z = (p - 1)(q - 1)$.
- 3) Escolher e (com $e < n$) tal que não possua fatores comuns com z , ou seja, e e z são coprimos.
- 4) Escolher d tal que $ed - 1$ seja exatamente divisível por z , ou seja, $ed \bmod z = 1$.
- 5) A chave pública é (n, e) e a chave privada é (n, d) .

Já a parte da criptografia e da descriptografia é feita da seguinte maneira: obtém-se os dados (n, e) e (n, d) calculados anteriormente. Assim, para criptografar uma mensagem m basta calcular: $c = m^e \bmod n$. Enquanto para descriptografar c basta calcular: $m = c^d \bmod n$ [1].

$$m = \left(\underbrace{m^e \bmod n}_c \right)^d \bmod n$$

A título de curiosidade, é apresentada no Apêndice A uma demonstração formal, usando aritmética modular, que explica por que o RSA funciona.

C. Geração de Números Primos e Teste de Miller-Rabin

Como foi visto na seção anterior é extremamente necessário usarmos números primos para o funcionamento adequado do algoritmo do RSA, contudo a questão que fica é como vamos gerar esses números primos.

Os métodos para gerar esses números envolvem as seguintes etapas e técnicas principais:

- 1) Seleção Aleatória e Sementes Seguras: Os números primos devem ser gerados de forma aleatória e ter um tamanho suficiente para evitar padrões previsíveis que poderiam ser explorados por atacantes. Para isso, são utilizados Geradores de Números Aleatórios (RNGs) criptograficamente seguros, que fornecem sementes para iniciar o processo de geração [2].
- 2) Testes de Primalidade: Como não existe uma fórmula simples para gerar apenas números primos grandes, o método comum consiste em escolher um número aleatório (geralmente ímpar e de grande porte) e testar se ele é primo [2].

Para realizar o teste de primalidade usaremos o teste de Miller-Rabin que é um algoritmo probabilístico projetado para determinar se um número específico é primo ou composto [2].

O funcionamento do teste baseia-se em propriedades da aritmética modular que são inerentes aos números primos. O processo básico segue estas etapas [2]:

- 1) Escolha Aleatória: O algoritmo seleciona um número aleatório x dentro do intervalo de 2 a $m - 2$, onde m é o número que se deseja testar [2].
- 2) Cálculo Modular: Aplica-se a fórmula matemática $x^{m-1} \bmod m$ [2].
- 3) Avaliação do Resultado:
 - Se o resultado da operação for 1, o número m é considerado primo [2].
 - Se o resultado for diferente de 1, o número é classificado como composto [2].

D. OAEP (Optimal Asymmetric Encryption Padding)

Ainda é possível melhorar a segurança da nossa criptografia RSA para isso utilizaremos o OAEP que é um esquema de preenchimento frequentemente utilizado em conjunto com a criptografia RSA [3].

O objetivo principal do OAEP é reforçar a segurança de algoritmos de criptografia assimétrica [3]. Suas funções fundamentais incluem:

- Transformação em esquema probabilístico: O RSA tradicional é determinístico, o que significa que a mesma mensagem criptografada com a mesma chave resultará sempre no mesmo texto cifrado. O OAEP adiciona um elemento de aleatoriedade que converte esse processo em probabilístico [3].
- Prevenção de fugas de informação: Ele impede o desciframento parcial de textos cifrados, garantindo que um adversário não consiga recuperar nenhuma parte da mensagem original sem ser capaz de inverter a função de criptografia (a permutação de alcapão) [3].
- Segurança contra ataques: O esquema é projetado para ser semanticamente seguro sob ataques de texto escolhido (IND-CPA) e, especificamente no caso do RSA-OAEP, seguro contra ataques de texto cifrado escolhido (IND-CCA2) [3].

O OAEP funciona usando um par de “oráculos aleatórios”, funções G e H , antes de o dado ser submetido à criptografia assimétrica [3]. O processo de codificação segue estas etapas principais:

- 1) Preparação do Bloco de Dados (DB): Cria-se um bloco concatenando o hash de uma etiqueta opcional (IHash), uma cadeia de preenchimento de bytes nulos (PS), um byte separador (0x01) e a mensagem propriamente dita (M) [3].
- 2) Geração da Semente: Uma semente (seed) aleatória de comprimento fixo é gerada [3].
- 3) Mascaramento do Bloco: Utiliza-se uma Função Geradora de Máscara (MGF) que recebe a semente como entrada para gerar uma máscara (dbMask). O bloco de dados é então combinado com essa máscara através de uma operação XOR (maskedDB) [3].
- 4) Mascaramento da Semente: O bloco mascarado resultante é passado novamente pela MGF para criar uma máscara para a semente (seedMask). A semente original é então combinada com essa máscara via XOR (maskedSeed) [3].
- 5) Resultado Final: A mensagem codificada é formada pela concatenação de um byte inicial nulo, a semente mascarada e o bloco de dados mascarado [3].

E. Função Hash: SHA-3

Além disso, em nosso projeto para fazer o hash das mensagens foi escolhido a função SHA-3 (Secure Hash Algorithm 3) como algoritmo de hash criptográfico.

O SHA-3 (Secure Hash Algorithm 3) é o membro mais recente da família de algoritmos de hash seguro, baseado no algoritmo Keccak [4] [5].

O grande diferencial do SHA-3 em relação às funções hash anteriores (como SHA-1 e SHA-2) reside na sua arquitetura interna:

- Construção de Esponja (*Sponge Construction*): Enquanto algoritmos como o SHA-2 utilizam funções de com-

pressão bloco a bloco, o SHA-3 utiliza uma estrutura de "esponja". Nesse processo, a entrada é primeiramente "absorvida" para um estado interno e , em seguida, o hash final é "espremido" para produzir uma saída de comprimento fixo [4].

- Independência de Design: Ele foi projetado para ser fundamentalmente diferente do SHA-2 em sua lógica matemática. Isso é importante porque, se uma vulnerabilidade teórica fosse descoberta no design do SHA-2, o SHA-3 provavelmente não seria afetado pela mesma falha, servindo como uma alternativa robusta [4].

III. AMBIENTE EXPERIMENTAL E ANÁLISE DE RESULTADOS

A. Descrição do Cenário

O experimento foi realizado em um ambiente de desenvolvimento local, sem dependência de rede, focado na implementação e teste dos algoritmos criptográficos.

- Hardware utilizado: Computador pessoal.
- Software utilizado: Python 3.11 com biblioteca padrão e ambiente de desenvolvimento VS Code.
- Configurações realizadas: Criação de scripts em Python.

B. Análise de Resultados

A implementação do nosso trabalho pode ser dividida em basicamente três núcleos.

1) *Geração de chaves e cifra*: A geração dos números candidatos a primos foi realizada utilizando números aleatórios de 1024 bits. Além disso, o bit mais significativo é forçado para garantir o tamanho mínimo desejado, enquanto o bit menos significativo é definido como 1 para assegurar que o número seja ímpar.

```
1 def gerarnumero():
2     numero = random.getrandbits(1024)
3     numero |= (1 << 1023)
4     numero |= 1
5     return numero
```

E depois para gerar o número primo é utilizada a função `gerarprimo()`, que aplica o teste de primalidade de Miller-Rabin, cujo algoritmo foi explicado na fundamentação teórica. A implementação completa do algoritmo encontra-se disponível em: [GitHub](#).

Após a geração dos dois números primos de 1024 bits, eles são utilizados na criação das chaves pública e privada do RSA seguindo os passos descritos na fundamentação teórica. Primeiramente, calculamos o valor $n = pq$, e o valor de $z = (p-1)(q-1)$.

Em seguida é utilizado o valor $e = 65537$ como expoente público. Antes da geração das chaves, é verificado se e e z são coprimos através do cálculo do máximo divisor comum.

O número é escolhido por ser um primo de Fermat que possui uma escrita em bytes que permitem uma eficiência ótima para o encriptar os dados, realizando 16 exponenciações quadradas e 1 multiplicação para cada um dos bits, um do início e um do final da escrita em bytes.

Após essa verificação, é calculado o valor de d , correspondente ao inverso multiplicativo modular de e em relação a z , de forma que: $ed \bmod z = 1$

Assim, a chave pública é então formada pelo par (n, e) , enquanto a chave privada é composta pelo par (n, d) .

```
def gerarchave(primeiro, segundo):
    n = primeiro * segundo
    totiente = (primeiro - 1) * (segundo - 1)
    e = 65537

    if math.gcd(e, totiente) != 1:
        raise ValueError("erro_em_gerarchave")

    d = modinv(e, totiente)

    chavepub = (e, n)
    chavepriv = (d, n)

    return chavepub, chavepriv
```

2) *RSA utilizando o OAEP*: Como explicado anteriormente, o OAEP é um esquema de *padding*, ou seja, antes da aplicação do RSA são adicionados elementos aleatórios à mensagem utilizando funções hash. Dessa maneira, o RSA deixa de ser um algoritmo determinístico, em que uma mesma mensagem cifrada com a mesma chave sempre produz o mesmo resultado, passando a funcionar de forma probabilística e aumentando a segurança da criptografia.

Na implementação desenvolvida foi criado o arquivo `oaep.py`, responsável por toda a lógica do OAEP. Primeiramente, foi implementada a função `mask()`, responsável pela geração das máscaras utilizadas durante o processo de mascaramento do bloco de dados e da semente.

```
def mask(data, seed, mlen, hash_func=sha3_256):
    :
    txt = b''
    hash_size = hash_func().digest_size
    for i in range(ceil(mlen / hash_size)):
        c = i.to_bytes(4, "big")
        txt += hash_func(seed + c).digest()
    return bytes(map(xor, data, txt[:mlen]))
```

Inicialmente é criada uma sequência vazia de bytes chamada `txt`, que será utilizada para armazenar os blocos gerados pela função hash. Em seguida, é obtido o tamanho do hash, permitindo determinar quantas iterações serão necessárias para gerar uma máscara com o tamanho desejado.

Executa-se sucessivas aplicações da função hash concatenando a semente (`seed`) com um contador convertido para bytes. Cada resultado produzido é concatenado em `txt` até que a quantidade de bytes necessária seja atingida.

Ao final é realizada uma operação XOR entre os dados originais e a máscara gerada. Esse procedimento corresponde ao mascaramento utilizado pelo OAEP tanto para o bloco de dados quanto para a semente aleatória.

Na parte da criptografia, foi implementada a função `cifra_oaep()`, responsável pela preparação da mensagem antes da aplicação do RSA.

Inicialmente é calculado o tamanho do módulo RSA em bytes através da variável k . Em seguida são obtidos o tamanho da mensagem, tamanho do hash utilizado.

Posteriormente é calculado o limite máximo permitido para a mensagem no OAEP. Esse limite depende diretamente do tamanho da chave RSA e do tamanho da função hash utilizada.

Em seguida é calculado o tamanho corresponde ao hash de uma etiqueta vazia, utilizada pelo padrão OAEP e criada uma sequência de bytes nulos, responsável pelo preenchimento necessário para adequar o tamanho da mensagem ao bloco RSA.

Após isso é montado o bloco de dados *data block*, formado pela concatenação do hash da etiqueta, da sequência de preenchimento, de um byte separador $0x01$ e da mensagem original.

Gera-se uma semente aleatória utilizando a função `urandom()`, responsável por introduzir aleatoriedade no processo e transformar o RSA em um esquema probabilístico.

E o último passo consiste na aplicação da função `mask()` para gerar o bloco de dados mascarado (`masked_data_block`) e posteriormente a semente mascarada (`masked_seed`). Por fim, a mensagem codificada, no formato OAEP é construída pela concatenação de um byte inicial nulo, da semente mascarada e do bloco de dados mascarado.

Além disso, durante os testes observou-se uma limitação importante do RSA utilizando OAEP. Ao tentar cifrar mensagens muito longas ocorreu truncamento da mensagem decifrada. Após análise, verificou-se que o problema estava relacionado ao limite máximo de tamanho imposto pelo OAEP, calculado por: $limite = k - 2 * tam_{hash} - 2$.

Portanto, mensagens muito grandes não podem ser cifradas diretamente utilizando RSA-OAEP. Em aplicações reais, esse problema normalmente é resolvido através de criptografia híbrida, utilizando algoritmos simétricos como AES para cifrar os dados e RSA apenas para proteger a chave simétrica.

```
masked_data_block = mask(data_block, seed,
                          k - tam_hash - 1, hash_func)
masked_seed = mask(seed, masked_data_block,
                   tam_hash, hash_func)

return b'\x00' + masked_seed +
       masked_data_block
```

Por fim, a etapa de descryptografia foi implementada através da função `decifra_oaep()`, responsável por desfazer o processo de mascaramento e recuperar a mensagem original.

Inicialmente são recuperados o tamanho do módulo RSA em bytes e o tamanho do hash utilizado. Em seguida a mensagem codificada é separada em três partes: o byte inicial nulo, a semente mascarada e o bloco de dados mascarado.

Depois é aplicada novamente a função `mask()` para recuperar primeiro a semente original e depois o bloco de dados original.

Em seguida é localizado o byte separador $0x01$, utilizado para identificar o início da mensagem original dentro do bloco OAEP.

Por fim, a mensagem original é extraída e retornada pela função, concluindo o processo de decifração do OAEP.

```
def decifra_oaep(n, msg_cifrada, hash_func=sha3_256):
    k = (n.bit_length() + 7) // 8
    tam_hash = hash_func().digest_size

    _, masked_seed, masked_data_block = (
        msg_cifrada[:1],
        msg_cifrada[1:1 + tam_hash],
        msg_cifrada[1 + tam_hash:]
    )

    seed = mask(masked_seed, masked_data_block,
                tam_hash, hash_func)
    data_block = mask(masked_data_block, seed,
                      k - tam_hash - 1, hash_func)
    data_block_sem_hash = data_block[tam_hash:]
    idx = data_block_sem_hash.find(b'\x01')

    if idx == -1:
        raise ValueError("Formato_OAEP_inv_lido")

    msg = data_block_sem_hash[idx + 1:]

    _, msg = data_block.split(b'\x01')
    return msg
```

3) *Assinatura: Geração e Verificação:* No desenvolvimento do nosso trabalho para gerar uma assinatura e fazer a sua verificação foi criado o arquivo `signature.py`. Nesse arquivo foi implementado a função `hasher()` que lê um arquivo e calcula o hash do arquivo usando o SHA-3. Após esse procedimento usa a chave privada RSA (d , n) para “encriptar” esse hash criando, assim, a assinatura do arquivo. E por fim codifica essa assinatura em Base64 para gerar uma string legível.

Além disso, também foi criado a função `verificar_assinatura()` que verifica se a assinatura

```
def cifra_oaep(n, mensagem, hash_func=sha3_256):
    k = (n.bit_length() + 7) // 8
    tam_msg = len(mensagem)
    tam_hash = hash_func().digest_size

    limite = k - 2 * tam_hash - 2

    if tam_msg > limite:
        raise ValueError(f"Mensagem muito longa. Limite permitido: {limite} bytes")

    label_hash = hash_func(b"").digest()

    padding_string = b"\x00" * (k - tam_msg - 2 * tam_hash - 2)
    data_block = label_hash + padding_string + b'\x01' + mensagem

    seed = urandom(tam_hash)
```

corresponde ao arquivo usando a chave pública RSA (e, n). Para realizar esse procedimento a função lê o arquivo e calcula o hash do arquivo usando o SHA-3 e o converte para inteiro. Em seguida, abre o arquivo de assinatura decodifica o Base64 de volta para bytes e converte esses bytes em um inteiro usando a chave pública. E para finalizar compara o hash do arquivo com o hash do arquivo de assinatura se forem iguais, isso significa que o arquivo não foi alterado desde que foi assinado, e a assinatura foi realmente gerada com a chave privada correspondente.

IV. CONCLUSÕES

Dessa forma, conclui-se que o trabalho permitiu compreender, de maneira prática e conceitual, o funcionamento do RSA desde a geração das chaves até sua aplicação em assinaturas digitais. Além disso, observou-se as limitações do RSA em sua forma básica, devido a sua natureza determinística, o que motivou o uso do OAEP como mecanismo de preenchimento, embora a sua utilização tornou o processo mais seguro a sua implementação mostrou restrições relevantes quanto ao tamanho máximo das mensagens. Por fim, o desenvolvimento do gerador e verificador de assinaturas confirmou a utilidade do RSA na garantia de integridade e autenticidade de arquivos.

```

1 def hasher(arquivo,d,n):
2     # Read file:
3     with open (arquivo, "rb") as f: mensagem =
        f.read()
4
5     # Hash file:
6     hash = hashlib.sha3_256(mensagem)
7     hashbytes = hash.digest()
8     print (f"\n{hashbytes}_=")
9
10    # Cryptography Hash with OAEP RSA:
11
12    [...]
13
14 def verificar_assinatura(arquivo,
    assinatura_arquivo, e, n):
15
16    # Original file hash:
17    print (f"==_Original_file_hash:_=")
18    with open(arquivo, "rb") as f: mensagem =
        f.read()
19
20    hash_atual = hashlib.sha3_256(mensagem).
        digest()
21    print (f"\n{hash_atual}_=")
22
23    hash_atual_int = int.from_bytes(hash_atual
        , byteorder='big')
24    print (f"\n{hash_atual_int}_=")
25
26    with open(assinatura_arquivo, "r") as f:
        linhas = f.readlines()
27
28    [...]

```

REFERÊNCIAS

[1] James F. Kurose; Keith W. Ross, *Computer Networking: A Top-Down Approach*, 8ª ed., Pearson, 2021.

- [2] Ezz-Eldien A.; Ezz M.; Alsirhani A.; Mostafa A. M.; Alomari A.; Alserhani F. et al., “Desafios e soluções computacionais: geração de números primos para segurança de dados aprimorada”, *PLoS ONE*, vol. 19, n. 11, e0311782, 2024.
Disponível em: <https://doi.org/10.1371/journal.pone.0311782>
- [3] AcademiaLab, “Relleno de cifrado asimétrico óptimo”, Disponível em: <https://academia-lab.com/enciclopedia/relleno-de-cifrado-asimetrico-optimo/>.
Acesso em: 03 maio 2026.
- [4] Finst, “Funções hash criptográficas explicadas”, Disponível em: <https://finst.com/pt/learn/articles/cryptographic-hash-functions-explained>.
Acesso em: 03 maio 2026.
- [5] SSL.com, “O que é uma função de hash criptográfico?”, Disponível em: <https://www.ssl.com/pt/artigo/o-que-%C3%A9-uma-fun%C3%A7%C3%A3o-de-hash-criptogr%C3%A1fico/>.
Acesso em: 03 maio 2026.

APÊNDICE

Teorema 1. *Deve-se mostrar que: $c^d \bmod n = m$, onde $c = m^e \bmod n$.*

Demonstração. Para quaisquer x e y , vale que: $x^y \bmod n = x^{(y \bmod n)} \bmod n$, onde $n = pq$ e $z = (p - 1)(q - 1)$.

Então,

$$\begin{aligned}
 c^d \bmod n &= (m^e \bmod n)^d \bmod n \\
 &= m^{ed} \bmod n \\
 &= m^{(ed \bmod z)} \bmod n \\
 &= m^1 \bmod n \\
 &= m
 \end{aligned}$$

□